图形学物理仿真 Tutorial for Lab 0

Part 2: A Tour of Our Codebase Structure

本课程的代码框架继承自可视计算与交互概论课程并做了微调，这篇指南是对该代码框架的详细介绍，以下是一个简要总结。

How to Complete Your Code for Physics Simulation

在每次作业中，大家需要像 lab0 给出的效果那样，搭建若干模拟场景（即左上角的5个 cases），并提供键鼠交互或是通过按键修改参数，给出物理真实的模拟效果。为此，需要完成三个层次的构建：

- 在模拟层面，需要大家编写物理模拟算法，模拟场景中物体的运动；
- 在交互层面，需要创建ImGui交互方式并与模拟场景进行绑定；
- 在渲染层面，需要将物理场景渲染成图片，提供丰富且便捷的可视化效果。

我们的代码框架为大家提供了大量封装，并完成了 OpenGL 渲染的相应初始化、管线搭建、ImGui 窗口创建等繁杂的工程，大家只需完成下列操作，即可达到课程要求并生成赏心悦目的模拟 demo：

1. 编写物理模拟算法：建议大家将其编写成一个类，存储场景中的物理信息（位置、速度等），并提供基于上一帧结果，计算下一帧物理量的接口；
2. 为每个模拟场景编写 `CaseXXX` 类，完成物理场景与交互的耦合，它需要：
   - 是 `Common::ICase` 的派生类，从而能被代码框架的其他接口所调用；
   - 存储一个物理模拟算法的实例，从而能够访问并控制场景中的物理量，进行渲染与交互；
   - 实现 `OnProcessInput` 函数，控制如何进行鼠标交互；
   - 实现 `OnSetupPropsUI` 函数，控制左侧交互栏的种类，并将其与场景中物理量进行绑定；
   - 实现 `OnRender` 函数，控制每一帧时场景应如何渲染在屏幕上；
3. 为整个应用编写一个 `App` 类，它整合所有的 cases，需要：
   - 是 `Engine::IApp` 的派生类；
   - 存储一个 `Common::UI` 的实例 `_ui` ，并为每个 cases类存储一个实例；
   - 编写 `OnFrame` 函数，执行 `_ui.Setup` 指令，即可完成 ImGui 窗口的创建
   - 在 `main` 函数中，执行 `RunApp` 函数，函数模版设为刚刚编写的 `App` 类，即可完成外部渲染流程的创建，

行文至此，大家可能还是觉得无从下手；不过，大家需要的许多操作与函数实现都可以照搬本次给出的样例代码！下面，我将以 lab0 示例代码的 CaseMassSpring 为例，介绍样例代码是如何达到上述效果的。

An Example Case: CaseMassSpring

（以下文件均位于 `src/VCX/Labs/0-GettingStarted` 目录下） `main.cpp` , `App.h` , `App.cpp` 中的写法可以直接照搬，只需将 `RunApp` 函数调用时的模版类改成自己定义的 `App` 类，在自己的 `App` 类中存自己的 `CaseXXX` 实例即可。这样， `RunApp` 函数会完成渲染流程的搭建， `_ui.Setup` 函数会完成ImGui窗口的初始化。

`CaseMassSpring` 类中存储的成员变量的作用是：

- 与物理过程有关的变量，包括 `_massSpringSystem` 控制物理系统的模拟以及物理参数的设定； `_stopped` 标志模拟是否停止
- 可通过交互改变的渲染参数，如 `_particleSize` 等；此外，相机 `_camera` 的位置可以通过交互改变，由 `_cameraManager` 提供的接口负责；此外， `_cameraManager` 也提供了键鼠交互的接口。
- 与渲染有关的变量：其中， `_frame` 代表当前渲染的帧， `_program` 控制渲染所需的着色器。 `_particlesItem` 与 `_springsItem` 分别负责点与弹簧的渲染，前者只需要渲染单独的点，因此属于 `UniqueRenderItem` 的实例；后者

还需记录顶点间的连接关系（线段，三角面片都属于此类），因此属于 `UniqueIndexedRenderItem` 的实例。在初始化时，我们需要设置正确的绘制类型 `PrimitiveType::Points/Lines` .

`OnSetupPropsUI` 函数的内容比较直接，通过为 ImGui 函数绑定相应的指针来操控相应的参数。如果想实现的效果在样例代码中没有涉及，大家可以浏览 vcx 代码库的各个 branch 或在网上进行搜索。

`OnProcessInput` 函数中，我们简单的调用了 `_cameraManager` 的交互函数，达到鼠标左键旋转/右键及 WASDQE 按键平移/滚轮沿连线方向平移相机位置的效果；

`OnRender` 函数中，我们首先 **根据当前帧与前一帧的时间间隔，计算仿真结果**。

```
if (! _stopped) _massSpringSystem.AdvanceMassSpringSystem(Engine::GetDeltaTime());
```

这是渲染和仿真效果耦合的核心，推荐同学们在后续的lab中，也按照该思路编写相应函数接口。后续的渲染流程比较繁杂，包括的步骤为：

- 设定当前帧的长宽，更新相机位置；
- 对于 `_particlesItem` ，渲染时需要更新其管理的顶点位置（使用 `UpdateVertexBuffer` 函数）；对于 `_springsItem` ，还额外需要顶点间的连接关系（使用 `UpdateElementBuffer` 函数，由于模拟过程中连接关系不变，因此只在 `ResetSystem` 中调用）
- 渲染开始时，需要做一些设置：启用当前帧，调用 `glEnable(GL_LINE_SMOOTH)` 为所画的线提供抗锯齿，设置点的大小以及线宽；
- 接下来就是使用着色器分别为点线完成渲染。渲染前，需要先为着色器传递参数，包括相机参数 `u_Projection` 与 `u_View` ，以及绘制的颜色 `u_Color` ，随后调用 `Draw` 函数进行渲染；
- 渲染完成后，恢复相应设置后函数返回。

以上我们简单介绍了大家需要完成的接口，希望通过上述讲解，大家能熟悉交互以及渲染需要完成的各步骤。以下是对一些环节的补充。

More on Rendering

- 在 lab1 中，我们需要涉及到刚体的模拟，立方体的渲染可以参考 CaseBox 中的渲染流程。样例代码中单独渲染了立方体盒以及框架，并且调用 `glEnable(GL_DEPTH_TEST)` 开启深度测试。
- 在 lab2 中我们需要完成粒子法流体模拟，样例代码中 CaseFluid 写了一份示例的渲染代码。我们将流体粒子渲染成小球，使用另一套着色器；该着色器的部分参数由 `BindUniformBlock` 函数与 `_sceneObject` 中的 `PassConstantsBlock` 相绑定。渲染时，将多个小球合并成一个 `ModelObject` 实例，再进行渲染。
- 在 lab3 中我们需要完成FEM模拟，对于软体内部的离散结构，大家可以选择像弹簧质点那样渲染各四面体的顶点和边（可以考虑是否要开启深度测试），或者渲染外层 mesh.

More on Interations

在 CaseMassSpring 代码中，我们只是简单地调用了代码库中相机交互的方法。如果大家希望设计更丰富的交互方法，可以关注这几处的代码：

- 想要设计自己的键鼠交互的同学，可以参考 `OrbitCameraManager` 类中 `ProcessInput` 的写法，调用ImGui的相关接口判断键鼠状态。
- 在CaseBox中，我们设计了 `OnProcessMouseControl` 函数，它在 `OnRender` 的开头调用来改变场景，使得按住alt键后鼠标左键可以平移物体位置。大家可以关注一下 `OrbitCameraManager` 类中 `ProcessInput` 的写法中关于 `altKey` 的部分，它控制是要平移相机还是预计算出物体在世界坐标下应该平移的距离，再在 `getMouseMove` 函数调用时返回这一值。

```
if (movingByMouse) {
    if (ScreenSpacePanning) {
        _moveDelta -= q * glm::vec3(panLeft, panUp, 0.f) + panFront * front;
    } else {
        _moveDelta -= q * glm::vec3(panLeft, 0.f, 0.f) + panUp * camera.Up + panFront *
front;
    }
    _state |= StateMove;
}


if (! altKey) {
    if (rotating) {
        _spDelta.Theta -= delta.x * RotateSpeed * heightNorm * (glm::pi<float>() * 2);
        _spDelta.Phi -= delta.y * RotateSpeed * heightNorm * (glm::pi<float>() * 2);
        _state |= StateRotate;
    }
    if (wheeling) {
        _logScale += ZoomSpeed * wheel;
        _state |= StateDolly;
    }
}
```

大家在设计时，也可以用类似的方法修改对应的函数，来设计自己的交互方法。

写在最后

如果想要进一步了解 Lab 代码可以阅读这篇指南，也可以向助教提出你的疑问。

衷心欢迎大家对我们的课程和 Lab 设计提出自己的看法，*也许明年的 Lab 代码库就是你的作品哦*。

最后，这门课程由北京大学可视计算与学习实验室独家荣誉出品，这里有最前沿的可视计算研究，最宽松的学习氛围，最 nice 的学长学姐，欢迎大家来玩！详情请咨询课程助教~